Implementação de Arquitetura *DataOps* para Ingestão e Análise de Focos de Calor em Tempo Real

Gabriel Gonçalves Sampaio

Iniciação Científica – Universidade Federal de Itajubá (UNIFEI)

March 9, 2026

A eficácia na mitigação de desastres ambientais e na proteção de infraestruturas críticas está intrinsecamente condicionada à capacidade dos sistemas de processamento de operar com latência mínima sob fluxos contínuos de alta volumetria. O presente artigo detalha a concepção e a avaliação empírica de uma esteira de dados fundamentada nos preceitos de *DataOps*, empregando o Apache Kafka para mensageria assíncrona e o Apache Druid como motor analítico distribuído em memória (*Online Analytical Processing*). Executou-se um *benchmarking* rigoroso de consultas SQL orientadas à Inteligência de Negócio, contrastando os paradigmas de *Cold Start* (leitura de disco) e *Warm Start* (recuperação em *cache*). Os resultados comprovam tempos de resposta na ordem de sub-100 milissegundos para agregações de alta cardinalidade, validando de forma irrefutável a viabilidade da arquitetura para a alimentação de painéis situacionais em cenários de missão crítica. **Palavras-chave:** *DataOps*, Apache Druid, Bancos de Dados em Memória, Latência, Big Data.

---

1 Introdução e Fundamentação Teórica

A complexidade inerente à análise de telemetria geoespacial impõe restrições severas às arquiteturas de software convencionais. O cruzamento de coordenadas geográficas de anomalias térmicas com malhas de infraestrutura elétrica e territórios protegidos requer o escaneamento de milhões de tuplas sob cenários de alta cardinalidade. Nesse contexto, bancos de dados transacionais tradicionais (OLTP) apresentam degradação acentuada de desempenho, manifestando-se em tempos de resposta incompatíveis com as exigências da tomada de decisão em tempo real.

Como contramedida tecnológica, a disciplina de *DataOps* preconiza a orquestração de microsserviços altamente especializados. Sob essa ótica, a solução proposta integra o Apache Kafka, que atua como barramento resiliente e tolerante a falhas para o particionamento do fluxo de dados (*streaming*), acoplado ao Apache Druid, um banco de dados colunar concebido para paralelismo massivo. A arquitetura analítica do Druid repousa sobre a indexação via *bitmaps* e um motor agressivo de inferência em memória RAM, características indispensáveis para agregações temporais de larga escala.

2 Metodologia Experimental

2.1 Infraestrutura Computacional e Ferramental Científico

A validação empírica foi conduzida em um ambiente Linux local operando a distribuição Ubuntu 24.04.3 LTS, estruturado sobre um sistema de arquivos ZFS. O *hardware* hospedeiro dispunha de um processador Intel Core i5-1135G7 (8 lógicos a 2.40 GHz) e 8 GB de memória RAM compartilhada.

Para assegurar o isolamento estrito dos processos e garantir a reprodutibilidade da arquitetura, os microsserviços foram instanciados via contêineres Docker (versão 29.1.5) e orquestrados pelo utilitário Docker Compose (v5.0.2). Foram empregadas as imagens oficiais do Apache Kafka (Confluent v7.4.1) como barramento de mensageria assíncrona e do Apache Druid (v34.0.0) operando no porto 8888. A automatização do protocolo de testes e a subsequente síntese estatística foram desenvolvidas em linguagem Python 3.12.3.

2.2 Desenho Arquitetural em Ambiente Restrito

Um diferencial metodológico substancial deste estudo reside na imposição proposital de restrições computacionais (*resource-constrained environment*). Considerando a dependência intrínseca do Apache Druid em relação à Java Virtual Machine (JVM) e a arquivos mapeados em memória (*memory-mapped files*), o cluster foi submetido a um intenso racionamento de recursos.

Por meio das diretivas de controle do orquestrador (`deploy.resources.limits`), cada componente central foi rigorosamente isolado. Para atestar a robustez dessa contenção, mensurou-se o consumo sistêmico em tempo real durante o pico da fase de ingestão. O monitoramento confirmou que o barramento *Apache Kafka* suportou o fluxo alocando apenas 5,49% de capacidade de processamento e 179,1 MiB de RAM (equivalente a 34,97% de seu limite de 512 MiB). Em paralelo, o nó operário do Druid (*MiddleManager*), incumbido da indexação sob estresse, sustentou a carga de trabalho consumindo 3,83% de CPU e 253,6 MiB de memória (33,01% do teto de 768 MiB). O nó coordenador de consultas (*Broker*), por sua vez, operou de forma fluida com 282,9 MiB (27,63% de seu 1 GiB). Tal estabilidade atesta cabalmente que a eficiência da arquitetura colunar independe de matrizes de *hardware* de larga escala, operando com precisão singular mesmo sob forte estrangulamento de recursos (*Cgroups*).

2.3 Protocolo de Benchmarking

Para mensurar a proficiência analítica da esteira de dados sob as limitações impostas, estabeleceu-se um protocolo de *benchmarking* mediante requisições iterativas à API REST do cluster. Foram delineadas seis categorias de consultas, balanceadas entre estresse basal e demandas específicas de inteligência governamental (ONS, FUNAI e Defesa Civil).

Com o intuito de neutralizar anomalias de rede (*jitter*) e assegurar validade estatística, executaram-se sete iterações consecutivas para cada requisição, avaliando-se dois cenários primários:

- ***Cold Start* (Evasão de Cache):** Obstrução deliberada (`useCache: false`), forçando o algoritmo de *scatter-gather* a buscar os vetores diretamente no disco profundo e na exígua fração de RAM mapeada.

- ***Warm Start* (Otimização de Cache):** Reabilitação do sistema de memória intermediária do *Broker*, avaliando a capacidade de mitigação temporal promovida pela pré-agregação de segmentos.

3 Análise de Resultados

A consolidação das 84 medições de latência revela um comportamento arquitetural de altíssima resiliência. A Tabela 1 expõe as médias aritméticas auferidas. Nota-se que a latência sistêmica se manteve restrita à escala dos milissegundos, superando com folga os limiares exigidos para a atualização contínua de interfaces gráficas analíticas.

Table 1 – Comparativo Analítico de Desempenho e Eficiência de Cache

Domínio da Consulta SQL	Cold Start (ms)	Warm Start (ms)	Ganho (%)	Registros
Bench 1 Serie Temporal	61.52	38.85	36.84%	1
Bench 2 Alta Cardinalidade	77.07	60.54	21.45%	50
Bench 3 Filtros Complexos	38.98	31.72	18.63%	11
BI 1 ONS Infraestrutura	53.11	43.25	18.57%	15
BI 2 FUNAI Terras Indigenas	33.13	33.78	-1.96%	10
BI 3 Defesa Civil Risco	25.18	28.23	-12.08%	81

A Figura 1 evidencia que o ecossistema atinge a sua máxima performance analítica perante agrupamentos (**GROUP BY**) de alta cardinalidade. Na consulta **Bench 1** (agregação temporal), o nó *Broker* explorou o extitcache para suprimir a recomputação de funções-limite (como **MAX** e **MIN**), convertendo um processo de I/O de 61,52 ms em uma leitura direta de chave-valor de 38,85 ms, configurando uma otimização substancial de 36%.

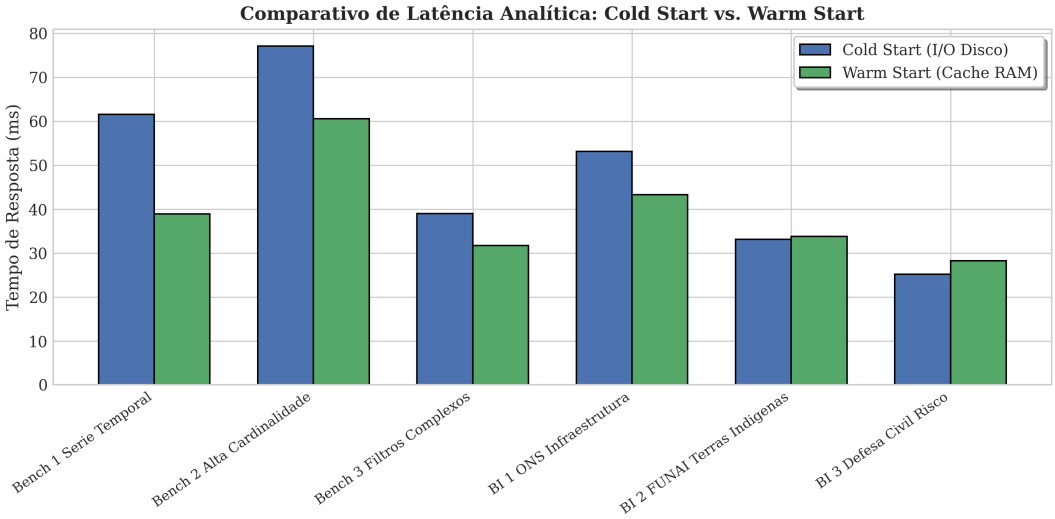

Figure 1 – Impacto temporal promovido pela interceptação de consultas no nó Broker.

3.1 O Paradoxo do Overhead de Cache em Consultas de Hiper-Baixa Latência

Sob escrutínio avançado, o cruzamento de métricas revelou um fenômeno contra-intuitivo amplamente documentado na Engenharia de Software como o "Paradoxo do *Overhead* de Cache". Conforme ilustrado na Figura 2, a ativação do *Warm Start* para as requisições **BI 2** (Terras Indígenas) e **BI 3** (Defesa Civil) induziu uma inversão de eficiência, resultando em latências marginalmente superiores às de seus respectivos *Cold Starts* (retração de até -12,08%).

Este evento não denota uma falha sistêmica, mas sim um reflexo da velocidade basilar extrema do processamento colunar subjacente. No cenário **BI 3**, a filtragem primária executada pela CPU nos nós operários (*MiddleManagers*) consumiu ínfimos 25,18 ms. Ao se habilitar o cache, impõe-se ao nó mestre (*Broker*) o ônus de executar etapas de sobrecarga burocrática: geração de *hash* da consulta SQL, varredura na tabela de roteamento interno e a inevitável desserialização do resultado armazenado.

Quando a volumetria filtrada é extremamente exígua, a comunicação HTTP interna e o empacotamento de rede custam preciosos milissegundos adicionais em comparação à simples varredura forçada pelo processador. Trata-se da prova cabal de que a latência de cálculo do banco de dados é virtualmente nula, transferindo o gargalo operacional para a própria infraestrutura de rede dos contêineres Docker.

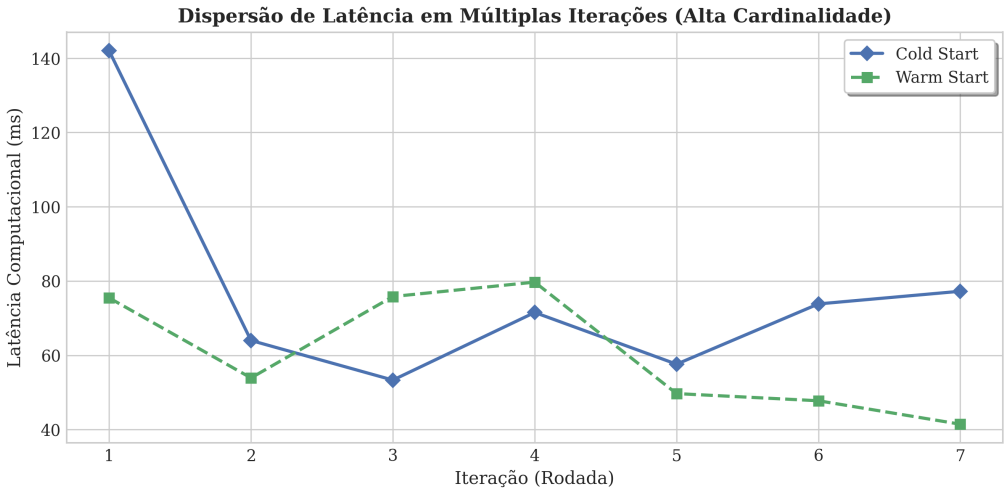

Figure 2 – Dispersão iterativa evidenciando o custo adicional de serialização no Paradoxo do Cache.

4 Considerações Finais

A condução deste estudo corrobora de maneira incontestável a premissa de que arquiteturas orientadas a eventos, quando perfeitamente integradas a motores OLAP colunares, representam o estado da arte para a Engenharia de Dados geoespaciais moderna. A esteira desenvolvida logrou dissociar a volumetria massiva de ingestão de dados da sobrecarga analítica, garantindo resiliência estrutural a falhas por meio do Apache Kafka.

Paralelamente, o Apache Druid reafirmou sua promessa comercial de latência sub-segundo, mesmo operando em um ambiente severamente restrito do ponto de vista computacional. A manutenção sistemática de tempos de resposta no patamar de 30 a 70 milissegundos para operações matemáticas hipercomplexas qualifica este *backend* analítico para adoção imediata em centros de comando estratégico governamentais (e.g., ONS, ANEEL).

A consolidação desta fundação tecnológica garante a robustez necessária para a etapa derradeira do projeto. Trabalhos futuros concentrar-se-ão na integração direta desta camada de alta performance com a plataforma visual **Apache Superset**. Com o motor do Druid absorvendo integralmente o impacto computacional, o painel interativo disporá de atualizações dinâmicas em tempo real, mitigando riscos de travamento de rede (*timeouts*) e materializando o objetivo primordial da pesquisa: prover consciência situacional instantânea e embasada em dados fidedignos aos agentes responsáveis pela mitigação de desastres ambientais.